Behavioral Drift in Autonomous LLM-driven Systems: A Survey of Detection Approaches and the Case for Deterministic Detection

Version 0.3 (2026-09-09).
Authors: Jürgen Eckel, Joerg Radehaus (KYDE).

Abstract

LLM-driven autonomous systems do not keep a fixed policy. Given the same task weeks later, they often take different actions. The literature labels this *behavioral drift*. The label is too coarse: it covers at least seven phenomena with different causes, different detection cost, and different legal effects.

We review eight recent detection papers. They range from composite stability scores and a non-identifiability theorem to runtime governance stacks and cheap black-box detectors. Three points follow.

1. Online detection is already practical. Reported cost is under 10 ms per action at one observation point and about 164 ms per prompt at another. Independently, the field settled on divergence over tool-use distributions as the default signal.
2. Published accuracy numbers are not comparable. Metrics differ (detection rate, ROC AUC, delay, violation counts). No paper reports results by drift type. Every evaluation uses synthetic data or LLM-labeled ground truth.
3. The detection cores themselves are deterministic statistics. LLMs appear in evaluation and diagnosis, not in the hot path. That split is required. An LLM judge is an agent and is therefore subject to the same drift it is asked to measure. Models also change behavior when they infer that they are under test.

If a detector is meant to support operations or evidence, it must be deterministic and recomputable from operational records. LLM judgment belongs off the critical path, under human review. We close with a detectability matrix: seven drift types against what boundary call logs can and cannot establish.

1. Introduction

An LLM agent that runs for weeks does not stay put. Context grows. Memory persists. Vendors replace models. Goals compete with the environment. Delegation passes intent through several systems. The gap between certified or intended behavior and later behavior is now called behavioral drift.

The regulatory pressure is specific. Article 72 of the EU AI Act requires providers of high-risk systems to monitor performance over the lifecycle. Article 12 requires records that make that lifecycle reconstructable. An operator who cannot tell whether behavior changed cannot meet either duty.

The term still lumps unlike things together. Section 2 separates seven phenomena. They differ in cause, in what can be observed, and in detection cost. Mixing them produces two failures: people talk past each other, and a detector that works on cheap cases is treated as if it covered expensive ones.

Contributions:

1. A survey of eight detection approaches (Section 3), compared on data needs, online capability, and runtime cost (Section 5).
2. A detectability matrix (Section 4) that crosses the seven phenomena with evidence, reported performance, and observability from operational call records. We have not found a prior breakdown of detection performance by drift type.
3. An argument (Section 6): existing detection cores are already deterministic; LLM components sit in evaluation and diagnosis; that separation should be a design rule, because an LLM judge is itself a drifting system.

2. What drifts: seven phenomena, two taxonomies

2.1 Operational taxonomy

The split comes from the surveyed papers. Each reported deviation answers three questions differently: what drives it (assigned goal, growing context, misspecified reward, evaluation setting, another agent, a memory store, or the vendor); where it shows up (action stream, outcomes, or model internals); and whether it survives the session.

Those axes give seven classes. They do not collapse into each other, and they cover the surveyed observations. Primary sources sit in Table 2.

1. **Goal drift.** The system leaves its assigned objective during a task (Arike et al. 2025).
2. **Context decay.** Performance falls as context grows. The main failure is early stop, not confusion (Laban et al. 2025; Xia et al. 2026).
3. **Reward hacking.** The system hits the measured proxy, not the intent. The goal was wrong from the start (Çağatan and Zhao 2026).
4. **Deception.** Test-time behavior differs from operational behavior (Apollo Research 2024; Anthropic and Redwood Research 2024).
5. **Multi-agent drift.** Deviation spreads through a group. In particular, drift is inherited via transcripts passed from one system to the next (Menon et al. 2026). The right measurement point is the seam between systems.
6. **Persistent drift.** Memory or self-modification makes the change durable. An incident becomes state (Lin et al. 2026).
7. **Version drift.** The vendor swaps or updates the model. Behavior changes with no operator action (Chen, Zaharia, Zou 2023).

Rath (2026) uses a three-way split: semantic, coordination, and behavioral drift. Useful vocabulary, orthogonal here. Each of Rath's surface forms can come from more than one of the seven causes.

2.2 Legal taxonomy

Regulation needs a coarser cut (Nannini et al. 2026, arXiv:2604.04604):

- **anticipated adaptation** — planned, inside stated bounds;
- **continuous learning** — gradual shift from data the system sees;
- **emergent drift** — unplanned change that no one specified and that cannot be read off the code.

The legal hook is Article 3(23) AI Act, *substantial modification*. Past some point the deployed system is no longer the assessed system. Adaptation is expected. The question is traceability: can the operator show how and when behavior changed. Table 2 maps this as a column.

2.3 Drift versus hijack: prompt injection

The taxonomy is by phenomenon, not by cause. Prompt injection is a cause. It still needs an explicit slot, because it produces three of the seven phenomena and is none of them in the strict sense.

- **Injected and persisted.** An injection that survives the session through memory or self-modification is persistent drift (type 6). This is the best-studied case. Self-evolving-system papers show how a one-shot injection becomes standing behavior.
- **Injected at hand-over.** Current models often resist a direct attack and still pick up drift from a weaker predecessor's transcript. Inherited drift is injection riding the seam (type 5).
- **Injected in-session.** A successful injection flips the effective objective at once. The surface looks like goal drift (type 1). The goal-drift literature studies the other regime: slow, endogenous deviation under environmental pressure, with nudges that never issue a new goal.

Security work keeps the line for a reason. In the OWASP Agentic Security Initiative, goal hijack (ASI01) needs an attacker; rogue behavior (ASI10) is the category that does not. We use the same cut: **drift is endogenous and gradual; hijack is exogenous and abrupt.**

That cut is operational. It splits detection into two statistical regimes (Section 5). Sliding windows and exponential smoothing are low-pass filters. They are built for creep and slow on jumps. The one surveyed bound on detection delay assumes bounded drift and fails on abrupt shifts.

Boundary instrumentation has a structural advantage for injection. The cause itself crosses the observation point: the injected content arrives in a request; later behavioral change appears in subsequent calls. One can correlate suspicious inbound content at time t with a distribution shift after t from boundary records. Output-only views see the effect. Each vendor console sees only its own slice.

2.4 Relation to classical concept drift

The statistics are old. Sliding-window divergence, exponential smoothing, and change-point tests come from concept-drift work on data streams. CUSUM is Page (1954). Adaptive windowing is ADWIN (Bifet and Gavaldà 2007). Gradual versus abrupt drift is Gama et al. (2014), a decade before LLM agents.

Behavioral drift is not that literature under a new name. It applies the same statistics to a harder object. The monitored distribution is an action stream, not a feature or label stream. The measured system can act on the state that measurement depends on, and in the persistent case it can rewrite that state. That is why the baseline must be frozen and the record integrity-protected (Sections 3 and 6). The verdict also has regulatory weight, not just model-maintenance weight (Section 2.2). The older field supplies detectors. The agent setting supplies the requirements.

3. The detection landscape

Eight recent works, grouped by role.

Composite metrics. The Agent Stability Index (Rath 2026, arXiv:2601.04170) combines twelve metrics in four weighted families (response consistency, tool use, coordination, behavioral bounds) over rolling 50-interaction windows. Inputs are full interaction logs, including tool calls and parameters. No model internals. Empirical numbers are projections from simulation. Cite the metric design, not the figures.

Formal limits of enforcement. Fernandez (2026, arXiv:2604.17517) proves a non-identifiability result. Under a local observability assumption that covers almost all guardrails, schema checks, and policy engines, no measurable function of the enforcement signal reconstructs membership in the admissible behavior set. Deviation can grow without bound while the enforcement channel stays quiet. The proposed fix is an invariant measurement layer: Jensen–Shannon divergence between current and admission-time tool distributions, against a frozen snapshot, with a finite detection-delay bound under bounded drift. Two design points: measurement sits above enforcement; the baseline is frozen. Rolling baselines let a drifting system become its own reference (*reference contamination*).

Runtime governance. MI9 (Wang et al. 2025, arXiv:2508.03858) combines an agent telemetry schema, a conformance engine that compiles temporal policies to finite-state machines, and goal-conditioned drift detection. Baselines are per objective. Drift under a stable goal is treated as suspicious. Drift with a verified goal change is treated as adaptation.

Agent Behavioral Contracts (Bhardwaj 2026, arXiv:2602.22302) checks declarative contracts per action in under 10 ms and adds a leading indicator: Jensen–Shannon divergence between the observed action distribution and a calibrated reference, fired before an explicit violation.

The Agent Viability Framework (Marín and Chaudhary 2026, arXiv:2604.24686) describes a fail-secure, monotonically tightening pipeline with dual-channel KL divergence and bandit-tuned thresholds. Analytical only; empirical evaluation is left open.

Lightweight black-box detection. Nautilus Compass (Wang 2026, arXiv:2605.09863) scores each prompt against curated behavioral anchors with embeddings. About 164 ms per check on CPU. Held-out ROC AUC 0.83 on real coding-agent traces. The paper states its own limit: innocuous surface text is invisible to a black-box detector. Activation methods (Abdelnabi et al., SaTML 2025) are the white-box counterpart.

Recovery. El Hamraoui et al. (2026, arXiv:2608.14109) move diagnosis and recovery into a small graph-based RL model with specialized roles per node. Recovery quality depends on a trusted onset time. That assumes a detection layer with a usable timeline.

Measuring the phenomenon. Arike et al. (2025, arXiv:2505.02709) score goal drift against a baseline run with two numbers: commission (wrong actions taken) and omission (right actions not taken). Omission is consistently larger. For log-based detection that matters: missing expected calls are a stronger signal than extra wrong calls.

4. Detection performance: what is reported, and what is not split

Published figures cannot be lined up. Units differ. Ground truth is synthetic or LLM-labeled.

Table 1. Reported detection performance.

WORK	REPORTED FIGURE	GROUND TRUTH
MI9	99.81% detection rate, 0.012% FPR (baselines: OTel+OPA 93.98%, LangSmith+OPA 68.52%)	1,033 synthetic, LLM-generated scenarios; LLM judge
Nautilus Compass	ROC AUC 0.83 held-out (keyword 0.62, zero-shot SBERT 0.75)	real traces; LLM labels
Invariant measurement layer	detection within 9–258 steps of onset; enforcement fired 0 times	3 simulated scenarios; mock LLM
Behavioral Contracts	5.2–6.8 violations per session missed by baselines; drift bounded below 0.27	1,980 sessions; LLM judge primary, human annotation as anchor
Agent Stability Index	drift onset at median 73 interactions; thresholds definitional, no rate	simulation, self-labeled
Agent Viability Framework	none (analytical)	none
Graph-based RL recovery	qualitative recovery gains reported; diagnosis and recovery, not detection	public AppWorld benchmark; results reported qualitatively
Goal-drift evaluations	occurrence 0.25–0.93 by setting; occurrence, not detection	the behavioral score *is* the ground truth

The strongest-looking number is the weakest as evidence. Section 6 treats an LLM judge as an unstable instrument. MI9's 99.81% sits on exactly that instrument. We do not claim the number is false. We claim nobody can currently show it is true.

Full human annotation does not scale to the regime that matters: weeks of operation and thousands of actions. Two workable paths remain. For phenomena that need judgment: an LLM judge pinned by human labels on a sample, as Behavioral Contracts do, keeps a checked error bound at machine throughput. For the deterministic rows of Table 2: ground truth can be events already in the record—version changes, declared goal changes, planted canary tasks—so no judge is required.

No surveyed paper splits detection performance by drift type. Each paper picks one phenomenon, one metric, and self-made truth. Operators need type × data access × performance. That matrix is not in the literature. Table 2 is a proposed structure, filled with what current papers support.

Table 2. Detectability matrix.

DRIFT TYPE	LEGAL CLASS (NANNINI ET AL. 2026)	BEST AVAILABLE EVIDENCE	DETECTION RATE REPORTED?	DETECTABLE FROM BOUNDARY CALL RECORDS?
Goal drift	emergent, or continuous learning	occurrence scores (Arike et al. 2025); goal-conditioned baselines, synthetic (MI9: Wang et al. 2025)	none on real tasks	partial: needs a declared goal per run and an end-of-run check; strongest log signal is omission (expected calls that stop)
Context decay	anticipated to emergent	phenomenon measured (Laban et al. 2025; Xia et al. 2026); no detector benchmark	none	yes, cheap: abort and error rate vs. run length are in the log
Reward hacking	misspecification; outside the drift triad proper	exploit rates 0–13.9% per model (Thaman 2026); ~72% of exploit attempts include explicit justification in the reasoning trace	no runtime detector	no: intended outcome is deployment-specific; intent sits in the reasoning trace, which does not cross the boundary

DRIFT TYPE	LEGAL CLASS (NANNINI ET AL. 2026)	BEST AVAILABLE EVIDENCE	DETECTION RATE REPORTED?	DETECTABLE FROM BOUNDARY CALL RECORDS?
Deception / scheming	emergent	activation probes only (Abdelnabi et al. 2025); models increasingly detect evaluation and corrupt the measurement (Schoen et al. 2025)	none robust	no: black-box ceiling; must be scoped out
Multi-agent drift	emergent	coordination metrics in simulation (Rath 2026); inherited drift moves the measurement point to the seam (Menon et al. 2026); no benchmark	none	partial: visible if the seam itself crosses the boundary (tool and delegation routing); otherwise needs a process object above the call chain
Persistent drift	continuous learning	attack persistence up to 100% in self-evolving stacks; scanners catch 2.5% (Lin et al. 2026); those are attacker success rates, not detector rates	none	indirect: the lasting effect is a durable distribution shift in the log; direct memory inspection needs per-framework hooks
Version drift	raises reassessment questions directly	phenomenon shown (Chen, Zaharia, Zou 2023); no detector benchmark	none	yes, cheapest: model and version are fields on every record; segmentation is enough

Prompt injection does not get its own row. It appears three times: persisted form as type 6, hand-over form as type 5, in-session form as an abrupt mimic of type 1 (Section 2.3). A detector that claims coverage of “drift from injection” must say which of the three it means. For the in-session case it needs change-point statistics, not only windowed divergence (Section 5).

Two consequences. First, the types with deterministic, log-computable detectors—context decay, version drift, distributional drift, and countable goal-persistence failures (duplicate submit, early abort, false success, missing progress)—are the types for which credible numbers can exist at all. Second, the types whose ground truth would first have to be defined by an LLM (open-ended goal drift, deception) are exactly where the regress in Section 6 sits.

5. Real-time cost and complexity

Online detection is shown, not hypothesized. The shared architecture is the same in every surveyed system: cheap statistics on the hot path, expensive diagnosis off it.

METHOD	ONLINE?	COST PER EVENT	COMPLEXITY
Contract or predicate check	yes, per action	< 10 ms	linear in constraints and action vocabulary; incremental
KL or JS divergence over tool distributions, sliding window	yes, streaming	negligible	$O(V)$ per update, V = tool vocabulary; EMA $O(1)$
Temporal policy conformance (compiled FSM)	yes, per event	negligible	amortized $O(1)$ per event
Embedding-anchor score	yes, per prompt	≈ 164 ms CPU	$O(N \cdot d)$ after one encoder pass

METHOD	ONLINE?	COST PER EVENT	COMPLEXITY
Statistical process control, adaptive thresholds	yes	negligible	$O(W)$ per window
Deep statistical inference (bootstrap, HAC errors)	on demand	$O(B \cdot W)$, $B \approx 1000$	kept off the request path
LLM-as-judge	offline or batch	one model call	too slow and too expensive for the hot path; see Section 6
Activation probes	yes, white-box only	needs a forward pass	unavailable without model access

Two theory notes. The invariant layer proves finite detection delay under bounded drift; that assumption fails for abrupt shifts such as prompt injection. The contracts paper shows drift scores are incremental, so windows need not be recomputed.

Section 2.3 maps onto two regimes on the same signal. Windowed divergence with exponential smoothing finds creep and is built to damp jumps. Abrupt change needs change-point tests (CUSUM family, sequential probability ratio tests). Those tests are also deterministic, also cheap per event, and better on steps than on slow wander. A deployed detector should run both against the same frozen baseline. The change-point channel answers whether behavior jumped after a given input. The divergence channel answers whether behavior wandered over weeks. Both are recomputable from the record.

Practical split: distributional and rule-based detection belongs at the sensor, at single-digit millisecond cost. Calibration, heavy inference, and semantic diagnosis belong in a control plane off the request path.

6. Why the detection core has to be deterministic

Across the surveyed systems, LLMs occupy three roles. Mixing them is the error.

1. **Detection.** Absent. Every hot-path detector is deterministic statistics. If a model is in the path at all, it is a frozen embedding encoder doing a fixed forward pass, not an agent issuing a verdict.
2. **Evaluation.** Common. Most benchmarks are scored by LLM judges on synthetic scenarios. That is a convenience for ground truth and a validity threat the papers themselves note.
3. **Diagnosis and recovery.** Emerging. Small trained models classify drift and propose fixes after detection.

The split is required, for three reasons.

Regress. An LLM judge is an LLM agent. It is exposed to every row of Table 2. It can drift with context, change with vendor versions, and pattern-match as inputs grow. Using it as the detector replaces one unmeasured system with two.

There is direct evidence that monitors are themselves measurement objects. Anti-scheming training reduces covert behavior without removing it, while models become more aware of being evaluated (Schoen et al. 2025 / OpenAI and Apollo Research, arXiv:2509.15541). SHADE-Arena exists to measure how well monitor models notice sabotage (Kutasov et al. 2025, arXiv:2506.15740). A layer whose verdicts must hold for months cannot rest on a component whose stability is still an open empirical question.

Reproducibility. A drift verdict has effects: lower trust, tighter permissions, human escalation. Disputed verdicts must be recomputed. Deterministic statistics over an append-only operational record can be recomputed by a third party from that record: same log, same score. A judge call generally cannot. Model versions move. Sampling varies. The judge's provider logs only its own slice. Signed independent records raise the weight of what happened from assertion to demonstration. A detection layer should keep that property, not spend it.

Baseline. Frozen baselines are the field's answer to reference contamination. An append-only, hash-chained record turns every past window into an integrity-protected baseline candidate, with goal changes present as events. That closes the gap between the frozen-snapshot school and the goal-conditioned school: both baselines come from the same immutable history.

Three record assumptions, stated rather than implied.

- **Completeness.** Every consequential action crosses the instrumented boundary. An uninstrumented side channel—direct network path, tool outside the gateway—is invisible to any log detector.
- **Integrity and order.** The record is append-only and tamper-evident (hash-chained, signed). The timeline under a verdict cannot be rewritten by the system being judged.
- **Attribution.** Each record names the agent, model, and version that produced the call.

A break of these is not drift. It is instrumentation failure or attack and belongs to another threat model. A deployment that cannot establish them should not treat downstream scores as evidence.

Limit of the claim. Deterministic boundary detection is incomplete. Deception with harmless surface text, and drift whose only evidence lives in reasoning traces or memory stores, stay out of reach without model or framework access (Table 2). LLM judgment is still useful. It belongs off-path, in diagnosis, with a human reading the output. It should not be the system of record.

7. Conclusion and outlook

The field already built, without a common plan, the structure its reliability problem needs: deterministic detection cores, statistical baselines, LLM judgment at the edge. It has not (a) split detection performance by drift type, (b) evaluated on real operational data rather than synthetic scenarios, or (c) stated determinism as a requirement. This survey supplies a structure for (a) in Table 2 and argues (c). Filling Table 2 with numbers from multi-week operational records, instead of "none", is the next step. That is the subject of follow-up work on measuring drift from signed boundary call records alone.

Reference implementations of the Section 5 detector set—the two-regime distributional detector and the per-type detectors of Table 2—plus a synthetic validation harness, accompany this paper as standard-library Python (https://github.com/kydehq/behavioral-drift-detection). Every verdict is meant to be recomputable from a record stream.

One extension should tighten detection earlier: per-agent task models. Production agents spend most of their time on recurring work. The record already shows that recurrence: runs cluster by goal, call-sequence shape, and duration. Trace clustering and process discovery (van der Aalst 2016) yield a per-agent, per-task reference: expected call sequences, branching probabilities, duration envelopes. That is tighter than a global tool distribution.

Three gains. Conditioning on the identified task cuts variance, so smaller deviations become significant sooner. Sequence-level conformance against a task model catches order drift that bag-of-calls divergence misses. A single run can be scored against its task model, so detection moves from week-scale to run-scale.

Section 6 still applies. Task models are learned off-path, then frozen and fingerprinted like any other baseline. Runtime scoring stays a deterministic function of model and record. Relative to Section 3, this is the goal-conditioned approach one step further: the condition is mined from the record instead of declared.

Disclosure

A large language model was used for literature search, screening, and first-pass extraction of the figures in Tables 1 and 2. In line with Section 6, that use was tooling, not a source of record. Every citation, quotation, and number was checked by the authors against the cited source. The authors take full responsibility for the content. No LLM is an author of the claims or conclusions.

References

Abdelnabi et al. Get my drift? Catching LLM task drift with activation deltas. arXiv:2406.00799; IEEE SaTML 2025.

Anthropic, Redwood Research (Greenblatt et al.). Alignment faking in large language models. arXiv:2412.14093, 2024.

Apollo Research (Meinke et al.). Frontier models are capable of in-context scheming. arXiv:2412.04984, 2024.

Arike, Donoway, Bartsch, Hobbhahn. Evaluating goal drift in language model agents. AIES 2025; arXiv:2505.02709.

Bhardwaj. Agent behavioral contracts: formal specification and runtime enforcement for reliable autonomous AI agents. arXiv:2602.22302, 2026.

Bifet, Gavaldà. Learning from time-changing data with adaptive windowing. SDM 2007.

Çağatan, Zhao. Reward hacking in language model agents: revisiting AI safety gridworlds. arXiv:2606.15385, 2026.

Chen, Zaharia, Zou. How is ChatGPT's behavior changing over time? arXiv:2307.09009, 2023.

El Hamraoui, Jose, Bureau, Plana. A graph-based reinforcement learning framework for structured drift diagnosis and recovery in autonomous LLM agents. arXiv:2608.14109, 2026.

Fernandez. From admission to invariants: measuring deviation in delegated agent systems. arXiv:2604.17517, 2026.

Gama, Žliobaitė, Bifet, Pechenizkiy, Bouchachia. A survey on concept drift adaptation. ACM Computing Surveys 46(4), 2014.

Kutasov et al. SHADE-Arena: evaluating sabotage and monitoring in LLM agents. arXiv:2506.15740, 2025.

Laban et al. LLMs get lost in multi-turn conversation. arXiv:2505.06120, 2025; ICLR 2026.

Lin, Deng, Li et al. Safety in self-evolving LLM agent systems: threats, amplification, and case studies. arXiv:2606.23075, 2026.

Marín, Chaudhary. Governing what you cannot observe: adaptive runtime governance for autonomous AI agents. arXiv:2604.24686, 2026.

Menon, Saebo, Crosse, Gibson, Jang, Cruz. Inherited goal drift: contextual pressure can undermine agentic goals. arXiv:2603.03258, 2026.

Nannini, Smith, Maggini, Panai, Feliciano, Tiulkanov, Maran, Gealy, Bisconti. AI agents under EU law: a compliance architecture for AI providers. arXiv:2604.04604, 2026 (legal drift classes and traceability); EU AI Act Art. 3(23), Art. 12, Art. 72.

OpenAI, Apollo Research (Schoen et al.). Stress testing deliberative alignment for anti-scheming training. arXiv:2509.15541, 2025.

OWASP GenAI Security Project. OWASP Top 10 for Agentic Applications 2026 (ASI01: Agent Goal Hijack; ASI10: Rogue Agents).

Page. Continuous inspection schemes. Biometrika 41(1/2), 1954.

Rath. Agent drift: quantifying behavioral degradation in multi-agent LLM systems over extended interactions. arXiv:2601.04170, 2026.

Thaman. Reward hacking benchmark: measuring exploits in LLM agents with tool use. arXiv:2605.02964, ICML 2026.

van der Aalst. Process mining: data science in action. 2nd ed., Springer, 2016.

Wang. Nautilus Compass: black-box persona drift detection for production LLM agents. arXiv:2605.09863, 2026.

Wang, Singhal, Kelkar, Tuo. MI9: an integrated runtime governance framework for agentic AI. arXiv:2508.03858, 2025.

Xia, Wang, Huang, Liu. Diagnosing and mitigating context rot in long-horizon search. arXiv:2606.29718, 2026.